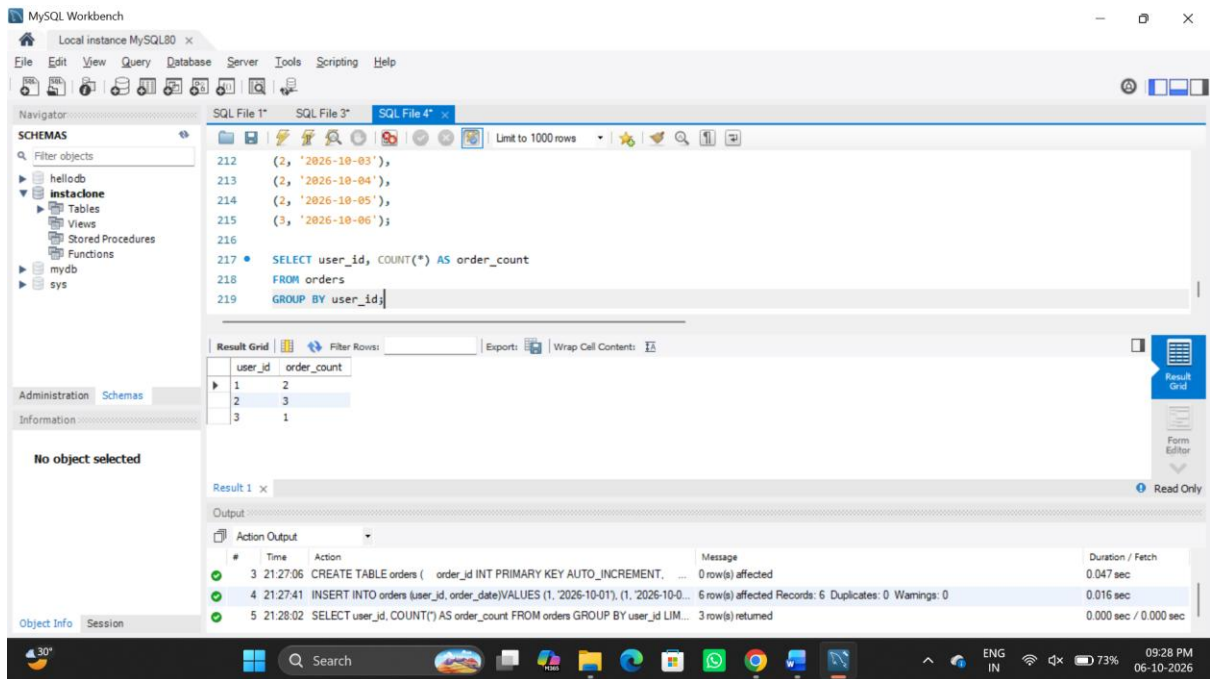


Session – 6

Q1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
212 (2, '2026-10-03'),  
213 (2, '2026-10-04'),  
214 (2, '2026-10-05'),  
215 (3, '2026-10-06');  
216  
217 • SELECT user_id, COUNT(*) AS order_count  
218 FROM orders  
219 GROUP BY user_id;
```

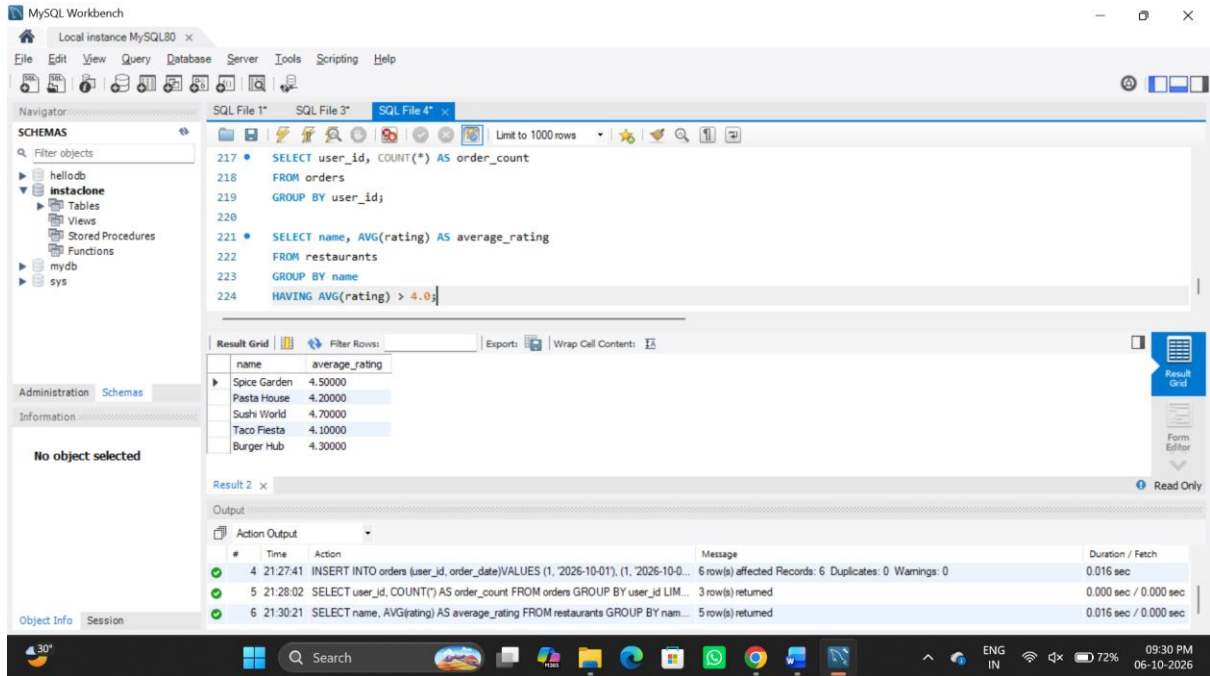
The Result Grid shows the output of the query:

user_id	order_count
1	2
2	3
3	1

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
3	21:27:06	CREATE TABLE orders (order_id INT PRIMARY KEY AUTO_INCREMENT, ...	0 row(s) affected	0.047 sec
4	21:27:41	INSERT INTO orders (user_id, order_date)VALUES (1, '2026-10-01'), (1, '2026-10-0...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.016 sec
5	21:28:02	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIM...	3 row(s) returned	0.000 sec / 0.000 sec

Q2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.



The screenshot shows the MySQL Workbench interface with three SQL files open. The first file contains a query to count orders by user ID. The second file contains a query to find the average rating for each restaurant. The third file contains a query to find restaurants with an average rating above 4.0.

```
217 • SELECT user_id, COUNT(*) AS order_count
218 FROM orders
219 GROUP BY user_id;
220
221 • SELECT name, AVG(rating) AS average_rating
222 FROM restaurants
223 GROUP BY name
224 HAVING AVG(rating) > 4.0;
```

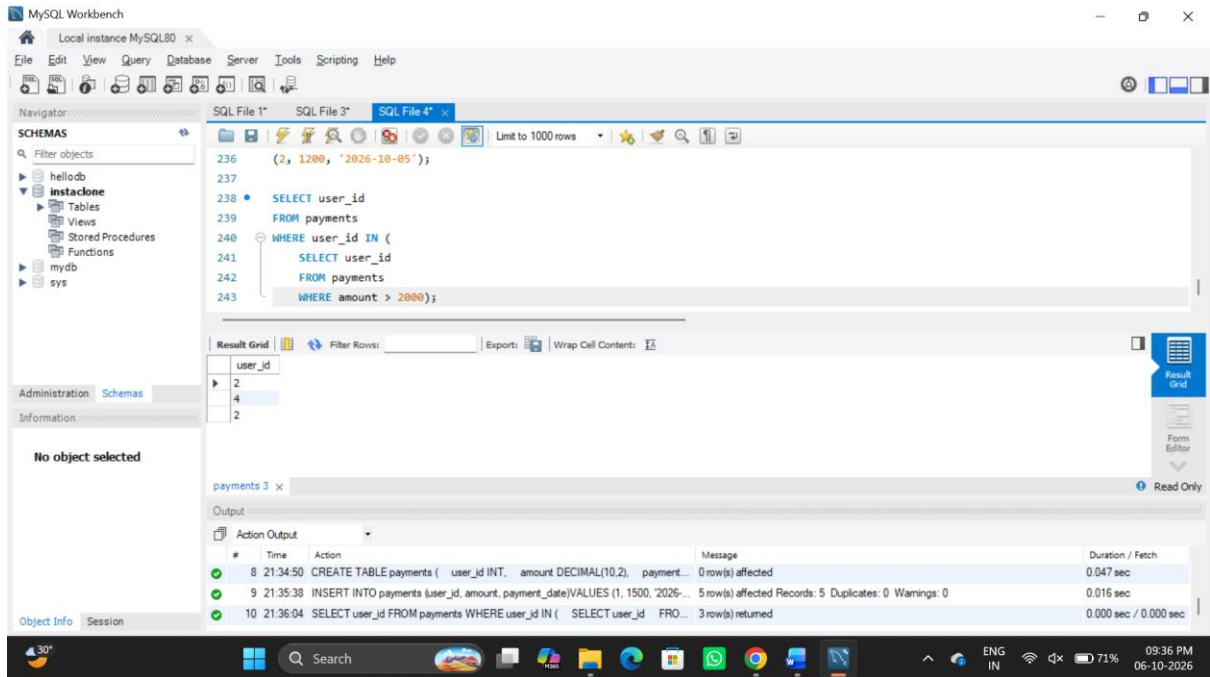
The Result Grid shows the output of the third query, displaying the names of restaurants and their average ratings.

name	average_rating
Spice Garden	4.50000
Pasta House	4.20000
Sushi World	4.70000
Taco Fiesta	4.10000
Burger Hub	4.30000

The Output pane shows the execution of the queries, including the insertion of data into the 'orders' table and the execution of the two SELECT queries.

#	Time	Action	Message	Duration / Fetch
4	21:27:41	INSERT INTO orders (user_id, order_date) VALUES (1, '2026-10-01'), (1, '2026-10-01')...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.016 sec
5	21:28:02	SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id LIM...	3 row(s) returned	0.000 sec / 0.000 sec
6	21:30:21	SELECT name, AVG(rating) AS average_rating FROM restaurants GROUP BY nam...	5 row(s) returned	0.016 sec / 0.000 sec

Q3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
236 (2, 1200, '2026-10-05');
237
238 SELECT user_id
239 FROM payments
240 WHERE user_id IN (
241     SELECT user_id
242     FROM payments
243     WHERE amount > 2000);
```

The Result Grid shows the output of the query:

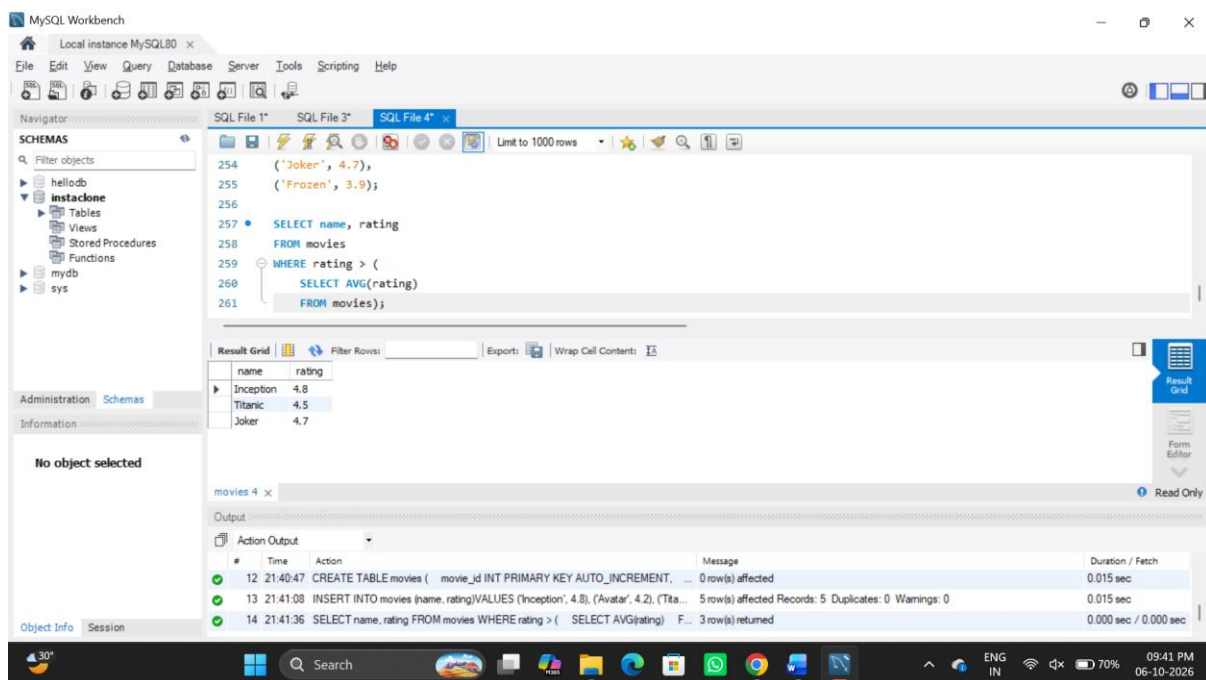
user_id
2
4
2

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
8	21:34:50	CREATE TABLE payments (user_id INT, amount DECIMAL(10,2), payment...	0 row(s) affected	0.047 sec
9	21:35:38	INSERT INTO payments (user_id, amount, payment_date)VALUES (1, 1500, '2026-...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
10	21:36:04	SELECT user_id FROM payments WHERE user_id IN (SELECT user_id FRO...	3 row(s) returned	0.000 sec / 0.000 sec

Q4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
254 ('Joker', 4.7),
255 ('Frozen', 3.9);
256
257 SELECT name, rating
258 FROM movies
259 WHERE rating > (
260     SELECT AVG(rating)
261     FROM movies);
```

The Result Grid shows the following data:

name	rating
Inception	4.8
Titanic	4.5
Joker	4.7

The Action Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
12	21:40:47	CREATE TABLE movies (movie_id INT PRIMARY KEY AUTO_INCREMENT, ...	0 row(s) affected	0.015 sec
13	21:41:08	INSERT INTO movies (name, rating) VALUES ('Inception', 4.8), ('Avatar', 4.2), ('Tita...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.015 sec
14	21:41:36	SELECT name, rating FROM movies WHERE rating > (SELECT AVG(rating) F...	3 row(s) returned	0.000 sec / 0.000 sec